

The *main.tf* contains the relevant infrastructure code and *variables.tf* contains the input variables for the *app* and the *vpc* directories.

Now have a separate repository which will house the entire code for all your environments. It will look like this:

```
> tree infra
infra
├── dev
│   ├── app
│   │   └── terragrunt.hcl
│   └── vpc
│       └── terragrunt.hcl
└── prod
    ├── app
    │   └── terragrunt.hcl
    └── vpc
        └── terragrunt.hcl
```

6 directories, 4 files

Each Terragrunt module specifies a Terraform block, which specifies the location of the Terraform module. Along with that, you will also have to specify an input attribute to define the input values for the module. This is how a sample *terragrunt.hcl* will look for the dev/app environment:

```
terraform {
  source = "git::git@github.com:psibi/infra-modules.git//app?ref=v0.1"
}

inputs = {
  instance_count = 2
  instance_type  = "m2.large"
}
```

If you want to go ahead and deploy just the *vpc* module for the dev environment, you can easily do that:

```
$ cd infra/dev/app/vpc
$ terragrunt plan
$ terragrunt apply
```

Simplify backend configuration

Problem being solved: Terraform doesn't allow variables or expressions in its backend configuration.

Terragrunt solves this without any hardcoding of values.

Taking the above infrastructure example, for dev environment *vpc* we want the following kind of state:

```
terraform {
  backend "s3" {
    bucket      = "my-infra-state"
    key         = "dev/vpc/"
    terraform.tfstate"
    region      = "us-east-1"
    encrypt     = true
    dynamodb_table = "infra-lock-table"
  }
}
```

Terragrunt allows you to solve this by putting a root *terragrunt.hcl* in the repository:

```
> tree infra
infra
├── dev
│   ├── app
│   │   └── terragrunt.hcl
│   └── vpc
│       └── terragrunt.hcl
└── prod
    ├── app
    │   └── terragrunt.hcl
    └── vpc
        └── terragrunt.hcl
└── terragrunt.hcl
```

6 directories, 5 files

Now you can define your backend configuration once in the root *terragrunt.hcl* file:

```
remote_state {
  backend = "s3"
  generate = {
    path      = "backend.tf"
    if_exists = "overwrite_terragrunt"
```

```
}
config = {
  bucket = "my-infra-state"

  key = "${path_relative_to_
include()}/terraform.tfstate"
  region      = "us-east-1"
  encrypt     = true
  dynamodb_table = "infra-lock-table"
}
}
```

All the heavy work here is done by the *remote_state* block and Terragrunt. The function *path_relative_to_include* block does the main work by giving unique backend keys for each of the folders. Inside *prod/app* it would have returned the value *prod/app*. This will make sure that you have unique key names in your bucket. For the above configuration to work, you also have to update the child *terragrunt.hcl* files with the following:

```
include "root" {
  path = find_in_parent_folders()
}
```

When to use Terragrunt

While Terragrunt is nice and useful, I recommend using Terraform if you don't have various environments to manage or if you are just starting out in your Infrastructure-as-a-Code journey.

Terragrunt solves real painpoints by simplifying backend configuration for Terraform when you have multiple environments with various modules to manage. I hope this post will make you think of and consider using Terragrunt when you write your infrastructure code next! **END** 🐧



The author is a senior engineer working at FPComplete.